

Cacheboard

Setup guide and outstanding tasks | github.com/the3dcoder/Cacheboard

What it is

A single-file dashboard (`index.html`) that tracks usage and limits across 12 platforms: AI quotas, cloud spend, website traffic, money, home lab, social, health and network security. An optional backend adds the few things a browser cannot do on its own.

Core principle: it never invents a number. Where no API exists, the card is marked Manual and explains why.

Setup

Step 1 - Open it (30 seconds)

Double-click `index.html`. No install, no build step. It opens with example cards so the page is not blank.

When ready for real data: **Settings > Cards > Remove all cards**, then add your own. *Restore examples* brings them back.

Step 2 - Add the easy credentials

These need no OAuth and no backend, so they work immediately:

Discord	Server ID only (enable Server Settings > Widget first)
Crypto	Nothing - CoinGecko is open. Just coin id + quantity
GitHub stars	Nothing for public repos. Just owner/repo
Anthropic	API key from console.anthropic.com
Cloudflare	API token (Zone > Analytics > Read) + Zone ID
YouTube	API key from Google Cloud Console (enable YouTube Data API v3)
Stocks	Free API key from finnhub.io/register
GitHub Copilot	Classic PAT with <code>manage_billing:copilot</code> scope

Step 3 - Only if you want OAuth cards or OpenAI

Azure, Twitch and Fitbit need a real web address - OAuth providers cannot redirect back to a `file://` path. The container gives you one and serves the dashboard too:

```
cd server
copy .env.example .env
(edit .env: set DASHBOARD_TOKEN to any long random string)
docker compose up -d
```

Then open `http://localhost:8787` and register that exact address as the redirect URI in each provider's console. In the dashboard, put the same URL and token into **Settings > Backend companion**.

Note: this machine has no Node, npm or Python installed, so 'npx serve' will not work. Use Docker, or serve the file from Home Assistant.

Step 4 - Home Assistant (optional)

In HA: profile > Security > create a Long-Lived Access Token. In Cacheboard: **Settings** > paste the base URL and token > **Test connection**. Then add a Home Assistant card and click **Browse** to pick entities - you never type an entity id.

Add this page's address under `http: cors_allowed_origins` in `configuration.yaml` and restart HA. Or skip CORS entirely by copying `index.html` to `/config/www/cacheboard/` and opening it from HA itself.

Outstanding tasks, highest priority first

#	Task	Why it matters / what it unlocks
1	Delete the old EarlDashboard folder	Close this session first. Everything in it is already on GitHub - verified nothing untracked. Work from Desktop\Cacheboard from now on.
2	Add the eight easy credentials above	Fastest path to a dashboard showing real numbers. No infrastructure needed.
3	Test the GitHub Copilot token	Open question: GitHub moved to AI Credits billing in June 2026 and the docs are ambiguous about personal accounts. Your token settles it. A 403 means personal accounts cannot self-serve yet and the card stays manual.
4	Move the backend to the home lab	Unlocks OpenAI API usage, stops Fitbit expiring every 8 hours, and runs the security checks. On a laptop it stops working when the lid closes.
5	Register the Azure app	One app registration unlocks three cards: cost, quota and resource health. Needs the SPA redirect URI set to whatever address you serve from.
6	Connect Home Assistant	Unlocks nine card types including lights, locks, covers, scenes, climate, media and camera snapshots. Waiting on your hardware being set up.
7	Set the security check targets	Add <code>SECURITY_HOSTS</code> , <code>SECURITY_TLS_HOSTS</code> and <code>SECURITY_HTTP_URLS</code> to <code>server/.env</code> . First port scan records a baseline; later runs report what changed.
8	Twitch and Fitbit OAuth apps	Both need an app registration with the redirect URI matching your served address. Fitbit also wants its refresh token in <code>server/.env</code> to stop the 8-hour re-connects.
9	Point the generic card at Pi-hole, Frigate, UniFi	Any device returning JSON works: give it a URL and a field path. Routing through Home Assistant is usually easier if it already has an integration.
10	Let real trend history build up	Sparklines and percentage changes only become useful after a few days of real readings. Nothing to do here except use it.
11	Decide on Instagram, Facebook, TikTok	Real APIs exist but need Business verification and app review - a multi-week process, not a coding task. Steps are written up in <code>NOTES.md</code> .

#	Task	Why it matters / what it unlocks
12	Optional hardening and extras	HTTPS on the backend (token currently travels in clear on the LAN); WebSocket push for instant Home Assistant updates; arp-scan / nuclei / IDS checks, which need Linux host networking the container does not have.

Known limits, so they are not surprises

- ChatGPT Plus / Team consumer quota has no API from any origin. That card is manual permanently. Only API-key spend can be read, and only through the backend.
- Bank and card balances (PNC, Amex) and utility bills are manual. Real aggregation needs Plaid, which requires a server-side secret and an application process.
- Pixel Watch health data is Android-on-device only. Manual entry. Fitbit works properly.
- X, Reddit, LinkedIn and Printables are blocked by the platforms themselves, not by missing code. Details are in NOTES.md.
- Credentials live in your browser's local storage and in server/.env. Neither is in git, which also means neither is backed up.

Full detail: README.md (overview), NOTES.md (per-platform setup and blockers), ROADMAP.md (what was built and why), server/README.md (backend and security checks).